

Multi-Channel Pulse Generator

User Guide

Last Updated: February 6th, 2026
Sebastian Quiros



Table of Contents

Multi-Channel Pulse Generator.....	1
Table of Contents.....	2
List of Figures.....	4
Preface.....	5
Project Resources.....	5
Acknowledgements.....	5
1. Introduction.....	6
1.1 Overview.....	6
1.1.1 Purpose of the System.....	6
1.1.2 What the System Accomplishes.....	6
1.1.2 How This Manual Is Organized.....	7
1.2 Introduction to Programming.....	7
1.2.1 Software Programs and Processors.....	7
1.2.2 Compiled Code and Machine Instructions.....	8
1.2.3 Hardware Defined Systems.....	8
1.3 Field Programmable Gate Arrays (FPGAs).....	9
1.3.1 What is an FPGA?.....	9
1.3.2 FPGAs vs. Microcontrollers.....	9
1.3.3 How is an FPGA “Programmed”?.....	10
1.3.4 The FPGA as a Custom Processor.....	10
1.4 Overall System Architecture.....	10
1.4.1 High-Level Data and Control Flow.....	11
1.4.2 Configuration Domain.....	11
1.4.3 Behavioral Domain.....	12
1.5 Instruction Decoding.....	12
1.5.1 What is an Instruction?.....	12
1.5.2 Operation Codes (OP Codes).....	13
1.5.3 Instruction Fields and Routing.....	13
1.6 System Memory.....	14
1.6.1 Purpose of Memory in the Pulse Generator.....	14
1.6.2 Memory Structure.....	15
1.6.3 Memory Read and Write Operations.....	15
1.7 System Timing.....	16
1.7.1 Clock Domain and Time Representation.....	16
1.7.2 Execution Flow.....	16
1.7.3 Phase Transformation.....	17
1.7.4 Retriggering and Validation.....	17
1.7.5 Trigger Handling and Inherent Latency.....	18
1.8 Serial Programming.....	18
1.8.1 Universal Asynchronous Receiver/Transmitter (UART).....	18
1.8.2 Bit Streams to Packets.....	19
1.8.3 Error Detection and CRC.....	20

2. Specifications.....	21
2.1 Timing Specifications.....	21
2.1.1 Time Base.....	21
2.1.2 Trigger/Output Latency.....	21
2.1.3 Delay Range.....	21
2.1.4 Pulse Width Range.....	21
2.2 Electrical Specifications.....	21
2.2.1 Trigger Input.....	21
2.2.2 External Clock Reference.....	21
2.2.3 Output Channels.....	22
2.2.4 Output Edge Characteristics.....	22
2.3 Communication Specifications.....	22
2.3.1 UART Interface.....	22
2.3.2 Protocol.....	22
2.3.3 Response Behavior.....	22
2.3 Power Specifications.....	22
3. Setup.....	23
3.1 FPGA Hardware.....	23
3.1.1 Quartus Prime Lite.....	23
3.1.2 Importing Project into Quartus.....	23
3.1.3 Assigning Pin Locations to the Verilog Module.....	25
3.1.4 Compiling and Synthesizing.....	27
3.1.4 Converting Program to Object File.....	28
3.1.5 Programming the Flash Memory.....	29
3.2 Computer Software.....	30
3.2.1 Pulse Controller Software.....	30
3.2.2 USB Driver.....	31
3.3 Physical Wiring.....	31
3.3.1 Materials List.....	31
3.3.2 Chassis.....	31
3.3.3 BNC I/O to Board.....	31
3.3.4 LEDs and Reset Button to Board.....	31
4. Usage and Basic Troubleshooting.....	32
4.1 Signal and Project Management.....	32
4.2 Channel Routing.....	33
4.3 Running the Experiment.....	34
4.4 Basic Troubleshooting.....	34
5. Further Discussion.....	35
5.1 Future Hardware.....	35
5.1.1 Full Portability.....	35
5.1.2 Sub Cycle Delays.....	35
5.2 Future Software.....	35
5.2.1 Experimental Modifiers.....	35
5.2.2 Simulation Pulse Previews.....	35

5.3 Future Architecture.....	36
5.3.1 Concurrent Programming.....	36
5.3.2 Sine Wave Rubidium Reference Clock.....	36
6. References.....	37

List of Figures

- **Figure 1.1** - High-Level Overview of the Pulse Generator system architecture
- **Figure 1.2** - Conceptual flow of software code running on a modern day CPU
- **Figure 1.3** - Simplified internal architecture of an FPGA
- **Figure 1.4** - Architectural Differences Between Microcontrollers and FPGAs
- **Figure 1.5** - Example 16-bit instruction to write '127' to channel 2 block 0's system storage
- **Figure 1.6** - Instruction Bit Mapping and Field Definitions
- **Figure 1.7** - Hierarchical 512-bit memory map illustrating dual 256-bit domains for delay and width parameters
- **Figure 1.8** - Pulse Generation State Diagram
- **Figure 1.9** - UART byte transmission format
- **Figure 1.10** - Packet framing format for all data sent to and from the generator module
- **Figure 1.11** - UART packet architecture illustrating the 9-byte sequence for a write delay command
- **Figure 3.1** - Quartus Prime Lite Edition download page on the Intel FPGA website
- **Figure 3.2** - Downloading the Pulse Generator project repository from GitHub
- **Figure 3.3** - The "Open Project" dialog within the Quartus Prime File menu
- **Figure 3.4** - Main project navigator in Quartus after successfully importing the hardware files
- **Figure 3.5** - "Remove Assignments" dialog configured to clear default pin locations
- **Figure 3.6** - The Pin Planner utility showing the Intel MAX 10 FPGA package layout
- **Figure 3.7** - Hardware pin assignment table for mapping Verilog nodes to physical DE10-Lite pins
- **Figure 3.8** - Task menu showing the hardware compilation and synthesis process in progress
- **Figure 3.9** - Task menu showing a successful compilation with green status indicators
- **Figure 3.10** - "Convert Programming File" menu configured for Internal Configuration mode
- **Figure 3.11** - Hardware Setup button used to access the Hardware Setup Menu
- **Figure 3.12** - Hardware Setup menu used to select the USB-Blaster programmer
- **Figure 3.13** - Final Programmer configuration screen prior to flashing the CFM memory
- **Figure 3.14** - Windows Defender SmartScreen warning for an unrecognized publisher
- **Figure 3.15** - Empty project state upon initial launch of the Pulse Generator software
- **Figure 3.16** - Software interface with multiple active signals and mapped channel assignments

Preface

This manual serves as a comprehensive guide on the design, operation, and replication of the digital pulse generator system. The aim of this project was to develop a high-precision hardware-defined alternative to software-based timing solutions using the Intel MAX DE10-Lite FPGA.

Project Resources

For the latest updates, access to the source code, and video demonstrations, please refer to the following:

- **GitHub Repository:** Contains the complete Verilog hardware source code, Quartus project files, and the Windows software client at:
<https://github.com/RealSebasTheFish/PulseGenerator/tree/main>
- **Video Tutorials:** Step-by-step tutorial on how to set up and use the system will be available at:
[TBD]

Acknowledgements

This system was originally designed for the Spartan Arty S7-50 before being adapted to the Intel DE10-Lite platform. Despite different boards requiring different programs to port the hardware, it is entirely plausible to find equivalent IP modules and adapt the hardware code to work on any similar FPGA hardware if so desired. This guide will be conducted on **Windows 11** with the **Intel MAX DE10-Lite FPGA**.

1. Introduction

1.1 Overview

1.1.1 Purpose of the System

This document aims to describe the underlying design, operation, and application of the multi-channel digital pulse generator implemented using the Intel MAX 10 FPGA DE10-Lite. The system was designed to provide a cost-efficient alternative to deterministic pulse generation based on an external trigger signal, with timing precision and repeatability suitable for non-critical timing in physics laboratory experiments.

The pulse generator produces four independent digital output channels, each generated based on a user-programmable delay and width. All parameters are relative to an external trigger, allowing the device to be used for general purpose applications such as detector gating, non-critical control systems, and event-synchronized control.

1.1.2 What the System Accomplishes

At a fundamental level, the system operates in two distinct phases:

1. Configuration
 - The user configures the desired pulse delays and widths using the software on a computer connected to the board
 - The configuration is sent to the FPGA over serial communication (UART)
 - The FPGA updates its internal state to preserve the configuration
2. Execution
 - After a start command is issued, the system listens for a trigger signal to be applied
 - On detecting the trigger, the system deterministically reacts to the stimulant with no software intervention
 - The pulses are then delayed and held for their configured timings controlled entirely by hardware

Once the system has been configured, it can respond to triggers repeatedly independent of the computer software, with the computer solely playing the role of powering the unit.

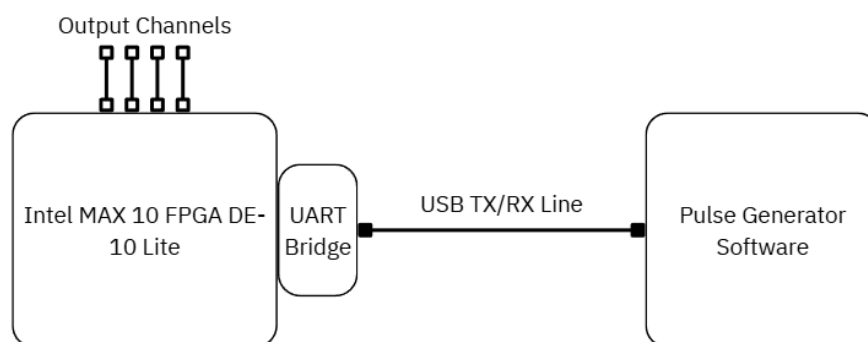


Figure 1.1: High-Level Overview of the Pulse Generator system architecture

1.1.2 How This Manual Is Organized

The remainder of this manual seeks to build a basic understanding of the system, as it can greatly benefit in understanding future replication of the system, the current limitations of the system under certain conditions, and how to troubleshoot any issues if they are encountered. The information is divided as follows:

- **Section 1** explains the conceptual foundations like programming, FPGAs, timing, and basic data/control flow
- **Section 2** outlines the specific electrical and timing limits behind the design
- **Section 3** covers the setup of the FPGA programming, wiring, and software linking
- **Section 4** explains how to operate the system once it has been set up according to all the previous sections
- **Section 5** discusses future possibilities and points of interest for further development of the unit

By the end of this document, the reader should have a general understanding of the inner workings of the system, and how its hardware defined nature differs from other software only based solutions.

1.2 Introduction to Programming

Before delving into how the pulse generator works internally, it is vital to establish an understanding of what exactly “programming” means when discussing electronic systems. Many identify programming at a high-level through languages such as Python or C, where code is written as a sequence of instructions to be executed by a processor. While it is entirely possible to develop a system such as this one with software, there are fundamental limitations to traditional computer programs which will be discussed.

This section introduces the distinction between the execution of software and hardware-defined deterministic behavior, which forms the basis of understanding the rest of the architecture

1.2.1 Software Programs and Processors

In a typical computer system, software is written as a sequence of instructions that are executed by the processor (CPU). The process is an iterative one:

1. Fetch an instruction from memory
2. Decode the instruction
3. Execute the instruction
4. Move onto the next instruction

Modern processors cycle through this process extremely quickly, but even then the instructions are executed one at a time sequentially, with timing depending on the many layers of abstraction such as operating systems, system architecture, and interrupts.

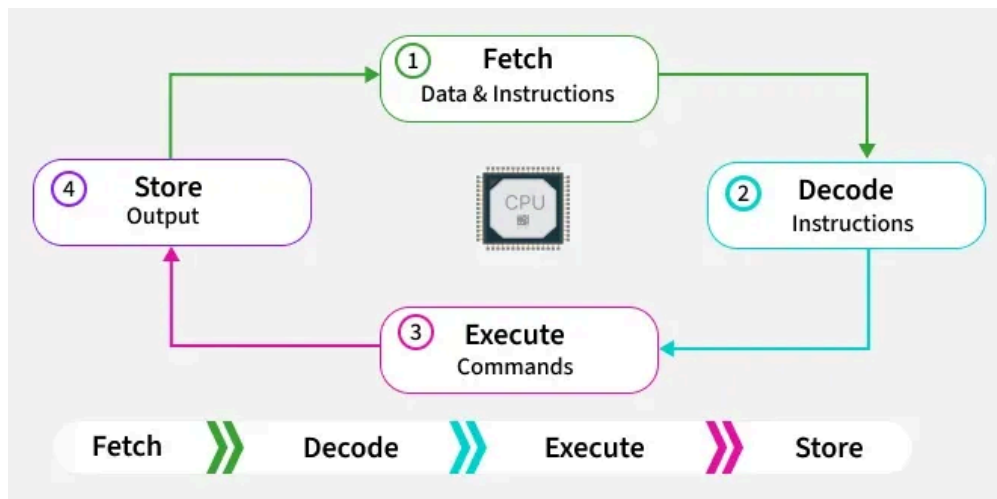


Figure 1.2: Conceptual flow of software code running on a modern day CPU [1]

1.2.2 Compiled Code and Machine Instructions

High-level programming languages are not directly understood or executed by hardware, instead, they are translated through several steps:

1. Source code (e.g. Python or C)
2. Compiled form
3. Machine instructions
4. Electrical signals in the processor

Machine instructions are ultimately resolved in the CPU to patterns of binary values to control the logic inside the CPU. The important thing to note is that the timing of these specific instructions are entirely removed from the developer's control. Even if two identical programs perform the same operations, their timing and execution can differ greatly between repeated iterations.

While this abstraction is useful for programming in a general sense, it presents a serious limitation for systems that require precise and deterministic execution.

1.2.3 Hardware Defined Systems

In contrast to software-driven systems, instead of developing a sequential series of instructions or conditions to be executed on a processor and memory, hardware systems define the physical behavior of signals and save states in physical storage.

In such systems:

- Logical components operate in a continuous time domain and in parallel
- The machine transitions state synchronously with a clock
- Timing behavior is all based on the physical propagation of signals

There are no instructions or code to be executed, but instead a system that continuously evaluates its own current states and inputs to advance to some next arbitrary state implied by the logic.

1.3 Field Programmable Gate Arrays (FPGAs)

1.3.1 What is an FPGA?

At an abstract level, an FPGA consists of three main components:

- **Configurable logic blocks:** Small electronic elements that can implement digital logic gates to define basic boolean functions and state storage
- **Programmable connections:** An interconnecting network between the logic blocks that allows them to be connected in arbitrary networks
- **Clocking and I/O resources:** Dedicated hardware for system clocks and physical input/output pins

When programming an FPGA, the user defines what logical components the logic blocks should represent, how the signals flow between them, and the intersections between the internal logical mesh and the external I/O (input/output) interface. Once configured, the FPGA behaves like a circuit, not as a running program.

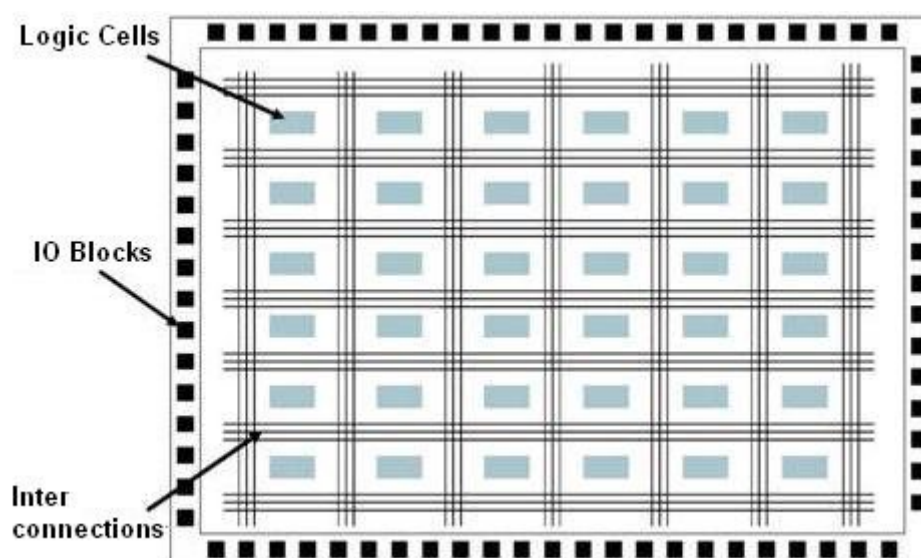


Figure 1.3: Simplified internal architecture of an FPGA [2]

1.3.2 FPGAs vs. Microcontrollers

While comparing the two systems can help bridge an understanding of how FPGAs work, it is important to understand their differences.

Component	Microcontroller	FPGA
Execution Model	Sequential program	Parallel hardware logic
Timing	Scheduled delays	Signal propagation
Determinism	Inconsistent	Very consistent
Flexibility	Proprietary components and	Fully configurable

	underlying architectures	
Applications	Control systems and computation	Timing and signal processing

Figure 1.4: Architectural Differences Between Microcontrollers and FPGAs

A microcontroller typically executes a single stream of instructions, relying on software interrupts and timers to approximate timing behavior. In contrast, an FPGA evaluates all timing simultaneously on each cycle of its clock. For systems where precise timing relative to stimuli is essential, their distinction becomes critical.

1.3.3 How is an FPGA “Programmed”?

Although development on an FPGA board is conventionally referred to as programming, the terminology can often lead to confusion when attempting to deploy a system on the gate array.

When writing software, you are describing what step should happen, which then its scheduling and execution is delegated to your computer’s processor. When configuring the FPGA, you must explicitly define hardware components and connect them, similar to a digital circuit board. All components such as memory/storage, physical data transport, states, and counters must be implemented using logical circuit components.

This is done using an HDL (Hardware Descriptive Language), and is synthesized into a bitstream uploadable to the board using software provided by the board’s manufacturer. In this case, the HDL used is Verilog, and the synthesizing software for the Intel MAX 10 DE10-Lite board is Quartus. These will be further described later in this manual in **Section 3**, when setting up the board and flashing the hardware to the board.

1.3.4 The FPGA as a Custom Processor

Rather than thinking of the FPGA as running code, it is more accurate to think of it as the processor of the system itself, similar to how an Intel processor is simply a circuit of electrical components. It is a processor that has been built from the ground up with this specific purpose in mind.

There is no general purpose CPU, no instruction memory, no applications, and no operating system to interface with. As **Section 1.4** will discuss, it is necessary to develop semantics and rules as to which the processor can be communicated with to make it configurable according to the user’s particular needs.

1.4 Overall System Architecture

To understand how the pulse generator behaves as a system in its entirety, it is helpful to step back and view the system first from a higher level, away from any individual hardware block or specific implementation. This section briefly introduces all the components of the system, and how they all interact through data and control flow signals.

At its core, the system consists of two tightly interconnected but distinct domains:

- The **configuration** domain, responsible for preserving the specific state of the machine in latch-based storage networks, and processing user requests to update the hardware's internal state
- The **behavioral** domain, responsible for generating the pulses based on its current state and the stimuli (trigger) received

1.4.1 High-Level Data and Control Flow

Conceptual flow of data:

1. The computer communicates with the FPGA with a stream of 1 and 0 bits using a USB UART serial interface (**Section 1.9**)
2. The stream is received, processed, and decoded into parallel signals to prompt the system to advance to the user defined configuration state (**Section 1.5**)
3. Once a certain configuration has been reached, the system enters a listening state waiting for a trigger (**Section 1.8**)
4. An external trigger causes the hardware to enter the user-defined timing domain to begin the hardware-controlled pulse generation (**Section 1.7**)
5. Output pulses are directed to the FPGA I/O pins for output to the BNC ports and LEDs

Once configuration is complete, the computer is no longer involved in timing or execution. All real-time behavior occurs entirely within the FPGA fabric.

1.4.2 Configuration Domain

The configuration domain of the system allows the user to describe what the system should do when some action occurs, without directly specifying when the event should take place. This domain includes:

- The serial communication interface (USB UART)
- Instruction processing and validation
- Data and control signal routing
- Internal latch-based storage for readable and writable user-defined system state memory

The user sends commands to modify the state of the hardware, such as:

- “Set the delay for channel 2 to x”
- “Set the width for channel 0 to y”
- “Start listening for triggers”
- “Stop and reset the system”

Each command directly modifies the state of the FPGA, not a software variable. This state is stored in hardware registers for later use when decoding what logical impact its current state has on its expected behavior. Because this configuration occurs through a fully controlled interface, the system behaves predictably across power cycles and isolated trigger events.

1.4.3 Behavioral Domain

Once configured, the system transitions into a state in its behavioral domain. This domain is responsible for:

- Monitoring the trigger input
- Physically keeping track of time elapsed into delay and width in terms of clock cycles
- Driving output pins with precise timings

In contrast to software systems, these systems aren't based on command based delays such as "wait" or "sleep", it is instead 4 concurrent counters being directly counted down by the rising edge of the clock signal being fed into all of them. The countdown sequence begins when the trigger input is detected, and output pins change state when each channel's respective counter expires. This system will be explored more in depth in **Section 1.7**.

1.5 Instruction Decoding

In order to make communication with the pulse controller possible, the system requires a structured way to interpret incoming information and map it onto changes in the hardware's internal state. This role is fulfilled by the **instruction decoding system**.

Instruction decoding is the process that translates a received command and parses it into control signals that guide data to where it needs to go to transition the hardware to its next target state. Importantly, the decoding system doesn't do any actions itself, but instead determines how the other existing hardware components in the system should respond to the current instruction and current state of the system.

1.5.1 What is an Instruction?

An instruction is a compact description of a desired operation. The set of all instructions possible on a given system is referred to as that system's **instruction set**. In this system, an instruction contains information about:

- What type of operation is being performed
- What channel the operation will apply to
- What portion of the memory will be configured or accessed
- Whether the desired action involves any auxiliary state transition besides storage augmentation

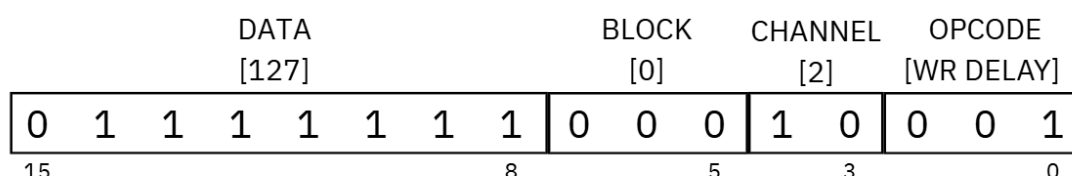


Figure 1.5: Example 16-bit instruction to write '127' to channel 2 block 0's system storage

These instructions are delivered from the computer software to the FPGA serially (discussed in **Section 1.9**), then reconstructed into a parallel signal that the hardware can process in a

single clock cycle. Instructions are defined as a group of 16 bits (2 bytes), capable of fully defining what action should be carried out.

1.5.2 Operation Codes (OP Codes)

The core of every system instruction is an operation code, commonly referred to as an opcode. When decoding an instruction, one starts from the least-significant bit (LSB), which is the bit furthest on the right, and considers this as bit 0. The first 3 bits in this case define the opcode. Below is a collection of all the opcodes in this system's instruction set:

- Writing a delay value [001]
- Writing a width value [010]
- Reading a delay value [101]
- Reading a width value [110]
- Start listening for trig [111]
- Stop listening for trig [011]

One interesting thing to note is that most systems must handle all input/output devices through these fixed sets of atomic instructions, as the processor is only built to handle these state changes. Although, as previously mentioned, since an FPGA defines things from a hardware level, it is possible to define any behavior according to a semantic the designer themselves decides, hence why I made a direct line that avoids any processing that connects the trig line directly to the timing core of the system (**Section 1.7**) to minimize latency. This highlights one of the key benefits of using an FPGA, and will be discussed alongside the trig bypass system later on.

1.5.3 Instruction Fields and Routing

Along with the opcode, an instruction contains supporting data to describe where and how the operation should be performed. These fields include:

- **Channel Selection:** Which one of the output channels the instruction wants to configure
- **Block Selection:** What component of the memory for that channel the instruction wants to configure (**Section 1.6**)
- **Data Payload:** What data will be used to configure the target

The exact information used by the instruction decoder to generate all the logical signals differs based on the opcode, but are all driven using **multiplexers**, a digital logic component that selects where data should be selected from or sent based on the value of certain signals. They are the heart of the decoding system, and allow the hardware to dynamically adapt its behavior based on the state of the selecting signals for the multiplexers in the circuit.

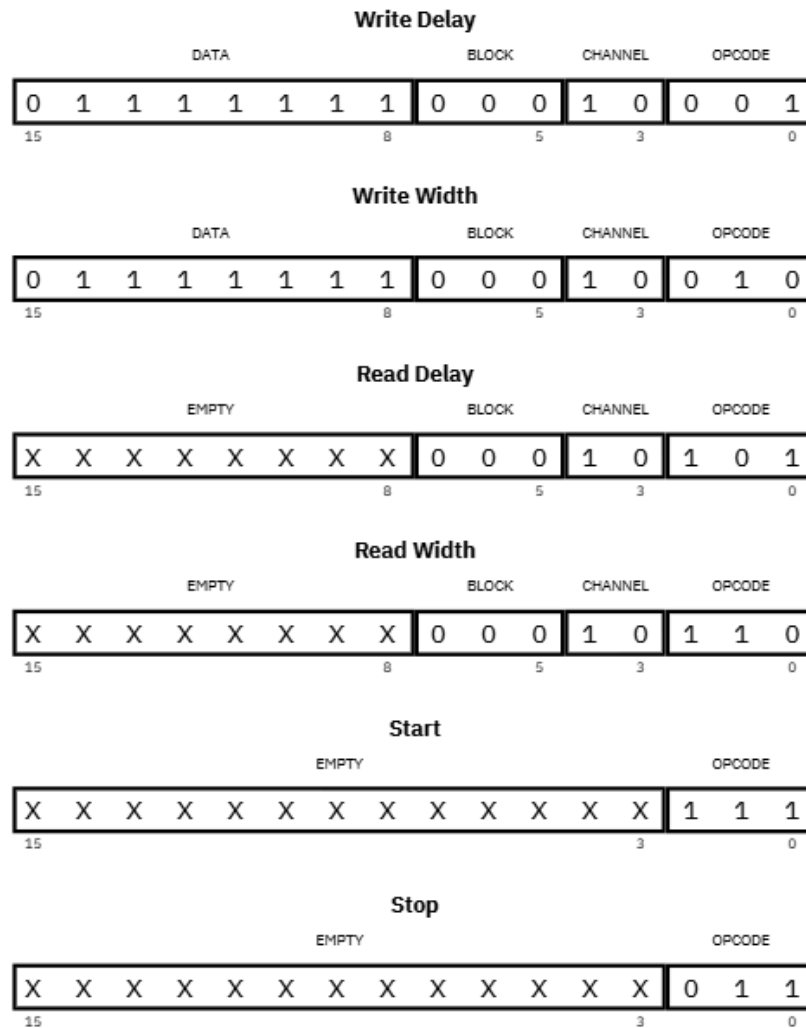


Figure 1.6: Instruction Bit Mapping and Field Definitions

1.6 System Memory

In order for the pulse generating system to behave predictably, it must have a means as to store the way it has been configured. This includes delays, widths, and control states, all which determine how the system reacts to a trigger. In the case of hardware-defined systems, this is accomplished through system memory implemented directly inside using the gate arrays in the FPGA.

Unlike software memory, whose implementation is abstract behind operating systems with dynamic management programs, system memory in this design is explicitly described as hardware latches and manually regulated by fixed behavior defined at synthesis time.

1.6.1 Purpose of Memory in the Pulse Generator

The role of the memory in the pulse generator system is to preserve its configuration state between trigger events. It does this by:

- Storing user defined delay and width values for each channel

- Retaining information while awaiting triggers
- Operating autonomously/statically without any dynamic behaviors being programmed in

Without memory, the system would need to constantly need to be receiving delay and width data, or have it be included as part of the trigger, which would defeat the whole purpose of a deterministic, low-latency pulse generator.

1.6.2 Memory Structure

System memory is organized in a channel-based system, reflecting the four output channels of the pulse generator. Each channel has its own section of dedicated storage, ensuring there aren't any conflicts between the channels and that the data retrieval process is efficient.

The system is capable of storing 512 bits of configuration data, segmented into 2 main blocks of 256 bits, one for delays, and the other for widths. Each segment is evenly divided into 64-bit chunks, one for each channel, allowing each to represent a large timing range with very fine relative resolution.

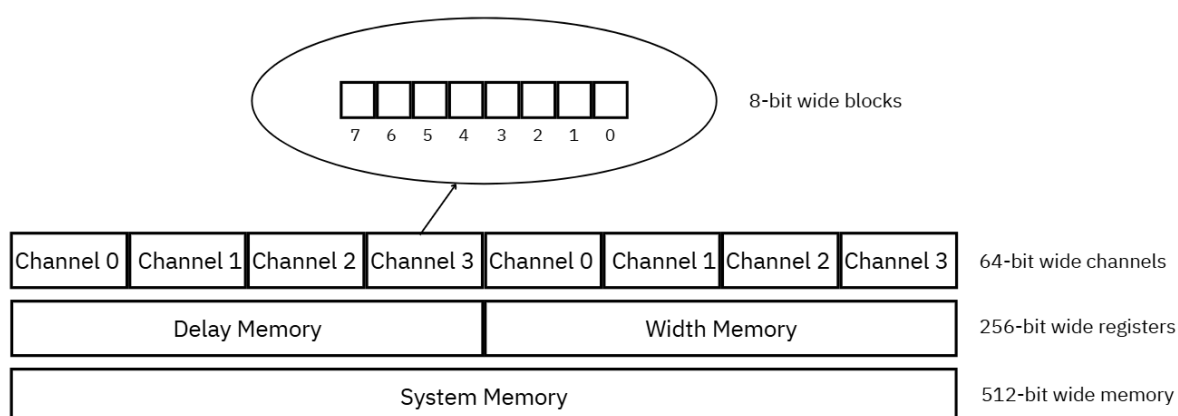


Figure 1.7: Hierarchical 512-bit memory map illustrating dual 256-bit domains for delay and width parameters

This structure allows the system to model timing values as 64-bit counters, where each unit of time is defined by the system clock. Instead of storing the time in seconds or other time units, it is stored internally as a count of clock cycles, which is converted to physical time later during pulse generation (discussed further in **Section 1.7**).

1.6.3 Memory Read and Write Operations

Memory access within the pulse generator is controlled entirely by the decoded instruction received through the serial interface. Instructions themselves do not directly manipulate the memory, but instead generate control signals in the instruction decoding module that determine how the existing memory should behave during a given clock cycle.

When an instruction that wants to write to memory is received, the instruction decoder enables a set of signals that routes the incoming byte of data to the target memory block and

channel segment. Over multiple instructions, a full 64-bit timing representation can be formulated by slicing the desired data into individual bytes (8-bit blocks). Similarly, read instructions may be reconstructed by buffering single bytes of data and combining them semantically on the software client.

1.7 System Timing

While the system memory defines what the pulse generator should do after a trigger is received and the system enters the behavioral domain, the timing system defines when the described events should occur. This section describes how time is represented and measured internally, and how deterministic behavior is achieved once the trigger event occurs.

Unlike traditional software-based systems, which approximate timings using scheduled delays or system interrupts, the pulse generator measures time directly using a hardware clock. All behavior is derived from the edges of this clock and implemented using synchronous logic that advances states on each clock cycle.

1.7.1 Clock Domain and Time Representation

Fundamentally, time inside the generator is represented atomically as discrete clock cycles. The system operates using a high-frequency internal clock that may optionally be slaved to a digital external reference clock through FPGA clocking resources. Each rising edge of the clock represents one unit of time.

Rather than storing delays in specific units of time such as seconds or milliseconds, the system stores them as the equivalent amount of clock cycles instead. This representation offers several advantages:

- Uniform handling of all timing values
- Exact repeatability between trigger events
- Removes potential overhead of unit parsing at trigger event
- Simple and efficient hardware implementation

If the system clock operates at some prescribed frequency f_{clk} , then each clock cycle corresponds to an increment of:

$$T_{clk} = \frac{1}{f_{clk}}$$

All delays and pulse width are therefore necessarily integer multiples of this atomic time unit. The smallest achievable timing resolution of the system is therefore exactly one clock period. There does exist a potential for sub-cycle timing procedures, which will be discussed later in **Section 5** of this report.

1.7.2 Execution Flow

Once the system has finished being configured and placed to its armed state through a START command, it continuously monitors the trigger input. When a valid trigger is detected,

the system goes from an idle state, into an active timing state which results in the following hardware state changes:

- 4 registers each 256 bits wide are enabled and opened access to the memory
- Delay and width counters (each 256 bits in total) are loaded into 2 of the registers from memory
- The delay and width counters are then copied into the remaining two registers
- Each channel is assigned a state register, and assigned a countdown phase based on its delay value

The system now has 2 copies of each the delay and width values for each channel stored. This is essential for efficiency, as it allows for the system to use one of the systems for physically counting down, and the other to reload the countdown registers on retrigger without needing to reassess the memory, significantly reducing latency on triggers. These counters operate independently and operate concurrently.

1.7.3 Phase Transformation

The initial phase of each channel after triggering and register loading is the delay phase. This phase defines how long the system waits after a trigger before asserting an output pulse. For each channel, the delay value is copied into the countdown register, and begins decrementing its stored value by one through binary arithmetic every time a rising clock edge is detected. After every operation, the state of the register is then evaluated to the state of 0, to which if it is found to be equal, signals the channel to advance to the next state on the following rising clock edge.

The next state is the width phase, where the output channel is asserted high, and the corresponding width counter for the channel begins to be decremented. The channel remains active while the register is greater than 0. For this reason, channels that have their width set to 0 will never have an output asserted high, serving essentially as a semantic enable/disable channel feature.

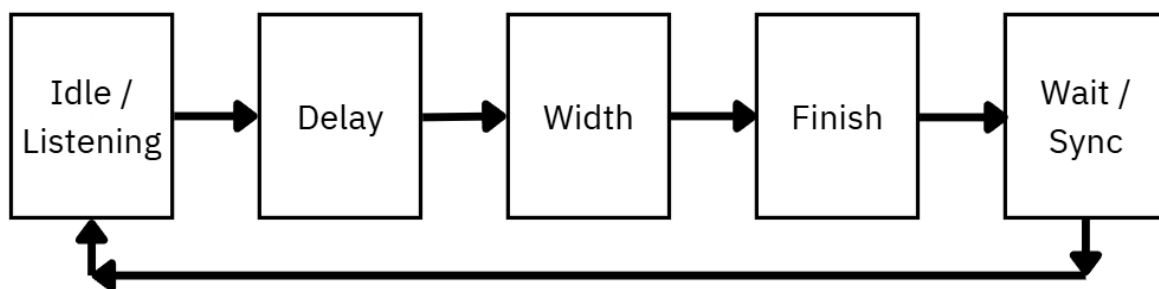


Figure 1.8: Pulse Generation State Diagram

1.7.4 Retriggering and Validation

Once a channel has completed its delay and width phase, it is assigned an intermediate finished phase, where it awaits for all other channels to also finish their corresponding states. Once all 4 channels are finished, they are then reset to the idle listening state, and await the next trigger to arrive.

If a trigger is asserted while the cycle is not yet done its execution, it will ignore the trigger and discard it as invalid. Additionally, it will assert the external “Trig Error” LED as high to indicate that the system is being triggered at too high of a frequency. The LED will be reset to low once all 4 channels are in their finished state and the system enters its idle state.

1.7.5 Trigger Handling and Inherent Latency

The external trigger input is routed through a dedicated hardware bypass line that connects directly to the timing core of the system, completely avoiding the delays caused by instruction decoding and serial communication. Because the FPGA implements a custom hardware-defined processor rather than executing software instructions, the trigger doesn’t need to be a command, and can instead act as a direct physical signal to the timing state. This design minimizes the latency and ensures deterministic and synchronized behavior.

Although the trigger signal initiates execution, it does not instantaneously begin the delay countdown at the exact moment when the signal transitions. Like all inputs on the FPGA, the trigger must properly be synchronized into the system’s clock domain to ensure consistency across trials. Additionally, the system must load the register values from memory, and instantiate the phase registers. As a result, there exists a small fixed latency before the counters begin countdown down.

This delay is deterministic and constant, as the memory always takes the same amount of time to access. Because of this, the latency does not introduce jitter or variability between pulses, and can instead be solved by introducing a fixed offset on the software side, aligning the trigger delay timing precisely with the desired programming from the user.

1.8 Serial Programming

While the pulse generator operates entirely autonomously once in the behavioral domain, it still requires a mechanism through which the user can communicate with it in its configuration domain. This is accomplished through a serial programming interface that allows the software to communicate the configuration data to the FPGA.

1.8.1 Universal Asynchronous Receiver/Transmitter (UART)

The system uses UART as its physical communication protocol, which is a simple and widely supported method of transmitting data as a stream of bits over a single data line, making it a good fit for configuration tasks. In UART communication:

- Data is transmitted one bit at a time
- Each bit is held for a specific duration of time defined by the baud rate
- There is no shared clock signal between the sender and receiver of the data
- Timing is derived from the agreed upon baud values

For this system, a baud rate of 115200 bits per second is used, meaning each bit is held for approximately 8.68 μ s.

To transfer 8 bits, or a byte, the following format is used:

- One start bit (logical low)
- The eight data bits
- One finish bit (logical high)



Figure 1.9: UART byte transmission format

The UART line is held at a logical high by default. This means that when a byte is transmitted, the start bit of logical low triggers a falling edge, which can be detected by the receiver as the start of a data byte. Then using the bit time of $8.68 \mu\text{s}$ as reference, can poll the line 8 times for all 8 bits of data sent, after which the line is then reset to the default by the finish bit and is ready for the next bit to be transmitted.

1.8.2 Bit Streams to Packets

Although UART is effective at transmitting individual bytes, the generator requires structured information in order to determine its internal state. To achieve this, the bytes are semantically grouped into packets, each holding a semantic value to help the receiver organize the data being transmitted. Each packet has the following information:

- Two synchronization bytes
- A length field
- A payload with a command
- A checksum to validate the packet

This fixed format allows for the system to have a robust capability to accept a wide variety of information precisely and efficiently, even in noisy signal environments.

Field	Size (bytes)	Purpose
Sync	2	Indicates the start of a packet
Length	1	Size of the payload
Operation Code	1	Operation to be performed
Channel	1	Target Channel
Block	1	Target Block

Data	1	Data
CRC	2	Checksum for error detection

Figure 1.10: Packet framing format for all data sent to and from the generator module through its serial interface

The two synchronization bytes (0x55, 0xAA in hexadecimal) allow the hardware to align itself with the incoming data stream in the case that any incoming data is lost or corrupted. Once these two bytes are detected, the system knows a formatted packet follows.

The serial decoder hardware parses the incoming packets, and constructs the corresponding 16-bit instruction, which is then forwarded to the instruction decoder module for execution. Complex configurations require several packets to be sent as a combination of the instructions available communicated through a series of these packet byte sequences. Importantly, these packets are buffered, so the executed instruction is pushed to the decoder in parallel for proper execution and synchronization with the internal clock.

Sync 1	Sync 2	Length	Opcode	Channel	Block	Data	CRC LSB	CRC MSB
0x55	0xAA	0x04	0x01	0x02	0x04	0x8C	0x71	0x39

Figure 1.11 - UART packet architecture illustrating the 9-byte sequence for a write delay command

1.8.3 Error Detection and CRC

To ensure the configuration data is not corrupted during transmission, each packet includes a CRC-16 checksum at the end. This checksum is computed by the sender using the data payload as an input into the function and included in the data before transmission, then recomputed by the receiver to check if the data integrity is preserved. If the two checksums match, and acknowledgement is sent back to the sender that the packet was successfully received. If they do not match, then the instruction is discarded, and a negative packet is sent back to indicate the need for retransmission.

For a complete and rigorous data transmission, the initial packet is constructed and sent, and the sender awaits for a response from the receiver. If a negative response, corrupted response, or no response is received after a certain timeout, the packet is retransmitted. This cycle continues until a positive acknowledgment is received, and the sender proceeds to the next configuration packet.

2. Specifications

2.1 Timing Specifications

2.1.1 Time Base

Internal timing clock: 200 MHz

Clock period: 5.0 ns

Timing resolution: 5.0 ns

2.1.2 Trigger/Output Latency

Trigger synchronization latency: 2 clock cycles (10.0 ns)

Internal pipeline latency: 1 clock cycle (5 ns)

Total fixed latency: 3 clock cycles + 5 cycles for general sync (40 ns)

Software compensation: Supported

2.1.3 Delay Range

Minimum delay: 50 ns

Maximum delay: $9.2 * 10^{10}$ s

Delay granularity: 5.0 ns

Delay reference: Trig rising edge

Delay jitter: ???

2.1.4 Pulse Width Range

Minimum width: 0 ns

Maximum width: $9.2 * 10^{10}$ s

Width granularity: 5.0 ns

Width reference: Delay expiration

Width jitter: ???

Width = 0 behavior: Output Suppressor

2.2 Electrical Specifications

2.2.1 Trigger Input

Logic standard: LVCMOS33

Input voltage range: 0V - 3.3V

Logical high (min): 2.0V

Logical low (max): 0.8V

Trigger edge: Rising edge

Minimum trigger width: 10.0 ns

Maximum frequency: 1 / full pulse pattern cycle

2.2.2 External Clock Reference

Logic standard: LVCMOS33

Signal type: Digital square wave

Voltage range: 0V - 3.3V

Accepted frequency: 10MHz

Note: Sine wave input not supported

2.2.3 Output Channels

Output logic standard: LVCMOS33

Output voltage (high): 3.3V

Output voltage (low): 0V

Drive strength: 16mA

Series resistor: 100 Ω (internal)

Effective output impedance: ???

Recommended load: ???

2.2.4 Output Edge Characteristics

Rise time (10-90%): ~25ns

Fall time (90-10%): ~25ns

Overshoot: +/- 0.4V

2.3 Communication Specifications

2.3.1 UART Interface

Physical interface: USB-UART

Baud rate: 115200

Data bits: 8

Stop bits: 1

2.3.2 Protocol

Frame sync: 0x55 0xAA

Payload length: 4 bytes

CRC: CRC-16-CCITT

Endianness: Little-endian

2.3.3 Response Behavior

Positive acknowledgement: 0x06

Error acknowledgement: 0x15

Cascading / Out of order responses: Not permitted

2.3 Power Specifications

Supply voltage: 5V (USB)

Unit I/O voltage: 3.0V - 3.3V

3. Setup

3.1 FPGA Hardware

The system is implemented on the Intel MAX FPGA (DE10-Lite Model). While the original prototype was developed on the Spartan Arty S7-50, it was found that this board was a suitable alternative based on the necessary specifications the boards shared in common.

3.1.1 Quartus Prime Lite

Every board has a specific program necessary to flash a hardware program onto it, which in this case, is Quartus Prime. There are several different versions for the program, but the simplest one to use without having to worry about licensing or other issues is [Quartus Prime Lite 22.1 STD](#). Ensure you download the version that includes the USB-Blaster drivers, usually accompanied by a “*Device Support Included*” indicator.

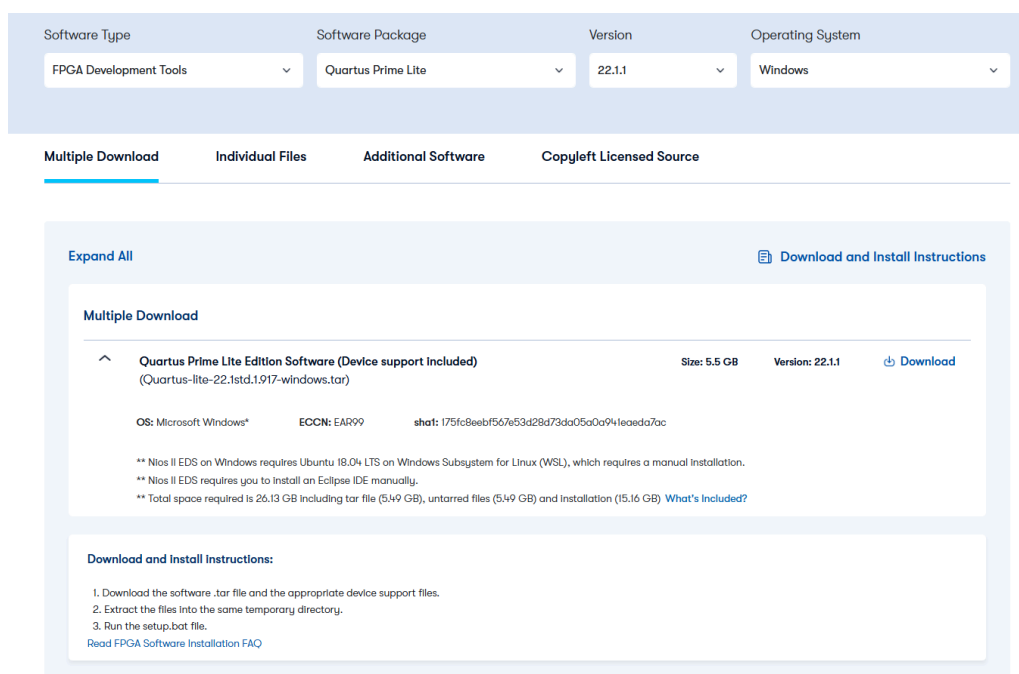


Figure 3.1: Quartus Prime Lite Edition download page on the Intel FPGA website

Please note that it is entirely possible to flash the board using a different version of Quartus Prime or a different operating system, but I will be using Quartus Prime Lite 22.1 on Windows 11 from now on as reference.

3.1.2 Importing Project into Quartus

First, begin by visiting the [Pulse Generator GitHub](#) page, and download the repository as a ZIP file. This can be done by pressing the green [< > Code] button, and selecting the ZIP archive option under the “*Local*” tab.

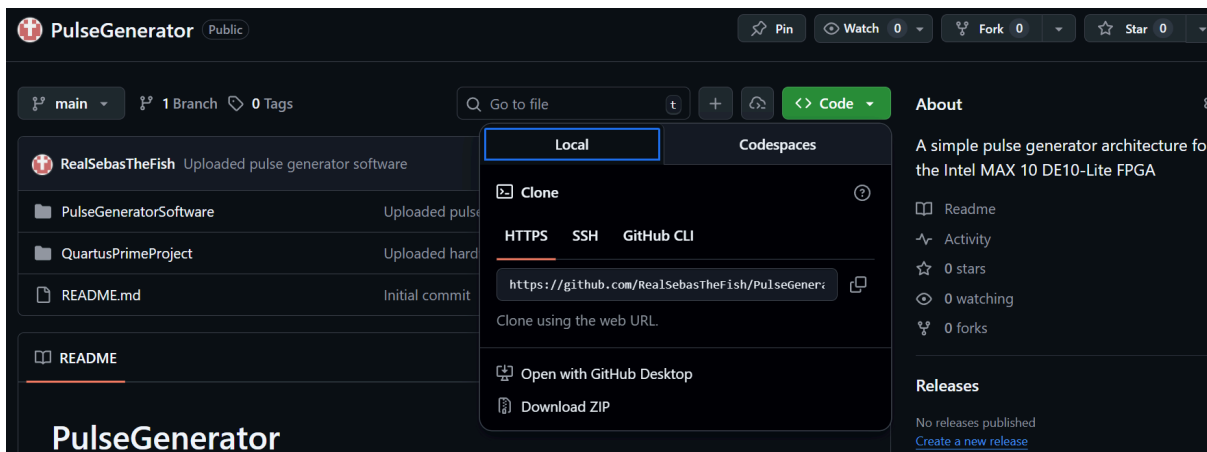


Figure 3.2: Downloading the Pulse Generator project repository from GitHub

Next, extract the zip file locally on the device being used to flash the hardware onto the board. This folder will include 2 directories, with the one named “*QuartusPrimeProject*” being the one we are interested in.

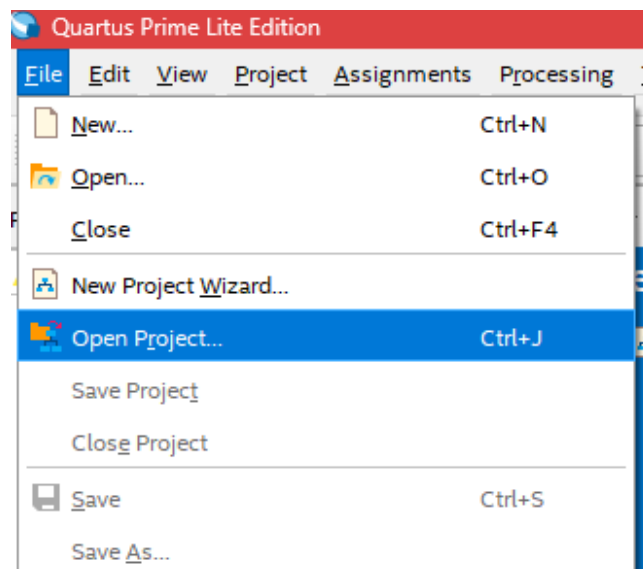


Figure 3.3: The “Open Project” dialog within the Quartus Prime File menu

Launch Quartus Prime Lite, and allow for all necessary dependencies and processes to conclude being imported on startup. After it finishes loading, press the “File” option, then the “Open project...” option in the dropdown. A file explorer window will open, in which you should then navigate to the folder you extracted from the Pulse Generator GitHub ZIP archive. Inside of the “*QuartusPrimeProject*” folder, you will find a file named “*PulseController.qpf*”. Select this file, and press “Open”, which will prompt Quartus to begin importing the full hardware project into the program. If any dialog box appears prompting you to only allow trusted sources, select the “Yes” option. After it finishes importing, the window should appear something like this:

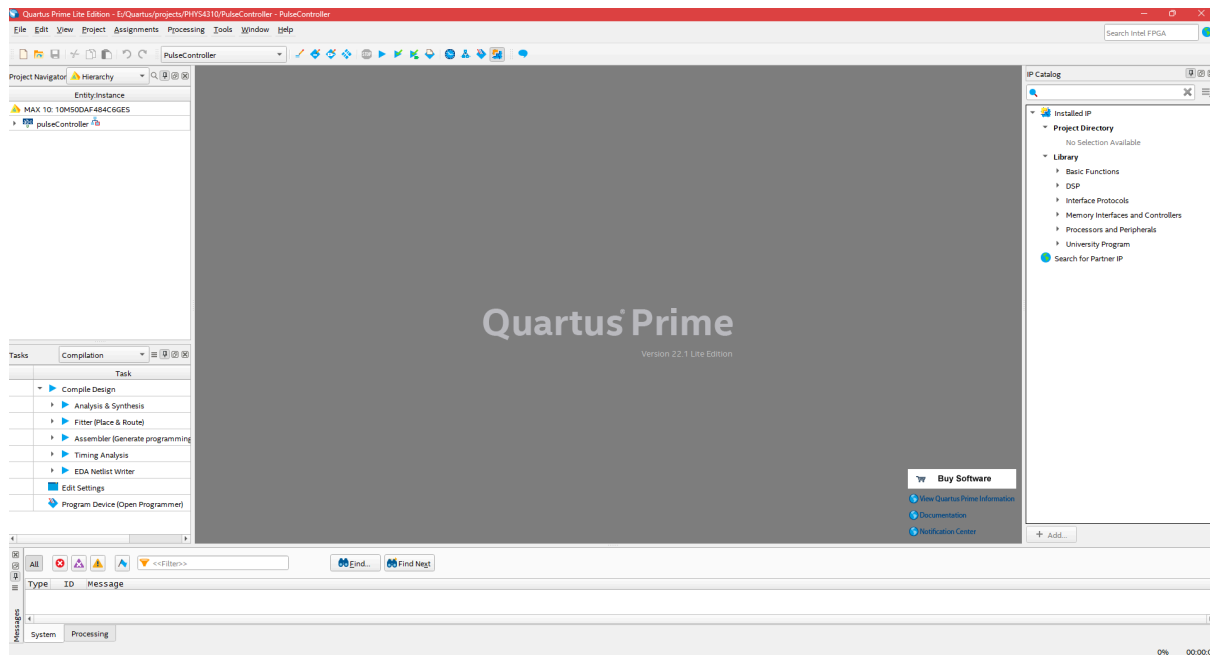


Figure 3.4: Main project navigator in Quartus after successfully importing the hardware files

3.1.3 Assigning Pin Locations to the Verilog Module

In order to properly interface the board's GPIO and other pins to the hardware module defined inside, we must discretely define which hardware pin corresponds to each of the module pins defined in the Verilog module.

Before we begin assigning the pins values, we must first clear the default values from the board. Select the "Assignments" option in the toolbar, and select "Remove Assignments" in the submenu. From here, select ONLY the "Pin, Location & Routing Assignments" option, then press "OK".

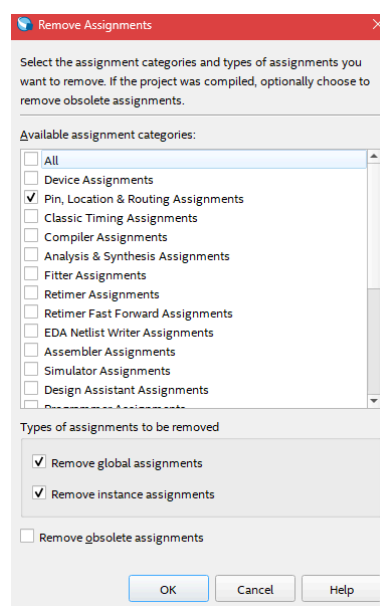


Figure 3.5: "Remove Assignments" dialog configured to clear default pin locations

Next, we can assign the proper values by going to the "Assignments" option in the toolbar again, and selecting "Pin Planner" this time. A new window will appear with all the hardware connections for the board displayed. The important part we will be using to assign the pin values is the table of the bottom with the headings "Node Name", "Direction", "Location", etc.

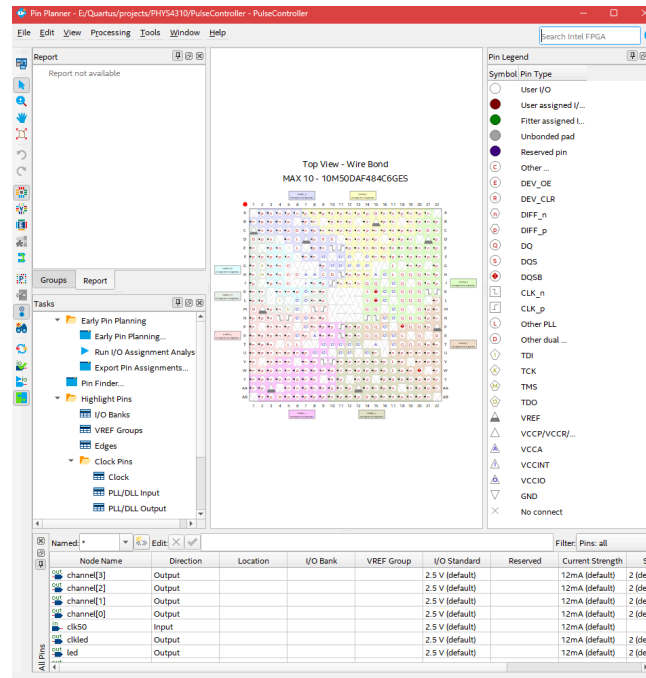


Figure 3.6: The Pin Planner utility showing the Intel MAX 10 FPGA package layout

The "Node Name" column corresponds to the name of the pin defined in the Verilog module. The "Direction" column defines whether the given pin is an input to the board, or an output from the board. Finally, the "Location" column, which should be currently empty for the Verilog pins, defines the corresponding physical location for the module pins on the physical board itself. To modify one of these values, double click on the empty cell corresponding to the Verilog module pin you are attempting to assign, and type in the desired pin location. Below is a list of all of the pin assignments necessary for the generator to function properly:

Node Name	Direction	Location
rb_clk	Input	PIN_V10
trig	Input	PIN_W10
usb_rx	Input	PIN_V9
usb_tx	Output	PIN_W9
channel[0]	Output	PIN_V8
channel[1]	Output	PIN_W8

channel[2]	Output	PIN_V7
channel[3]	Output	PIN_W7
clkled	Output	PIN_V5
startled	Output	PIN_W6
trigled	Output	PIN_W5
clk50	Input	PIN_P11

Figure 3.7: Required hardware pin assignments for the pulse generator mapped to physical location on the DE10-Lite

After all the pin assignments have been completed, you can close the pin assignment window, and return back to the main Quartus page.

3.1.4 Compiling and Synthesizing

Once the pin assignments are completed, we can now begin compiling and synthesizing our program into a bitstream to feed into the board. We start this process by double clicking the “*Compile Design*” button under the “*Task*” menu at the bottom left.

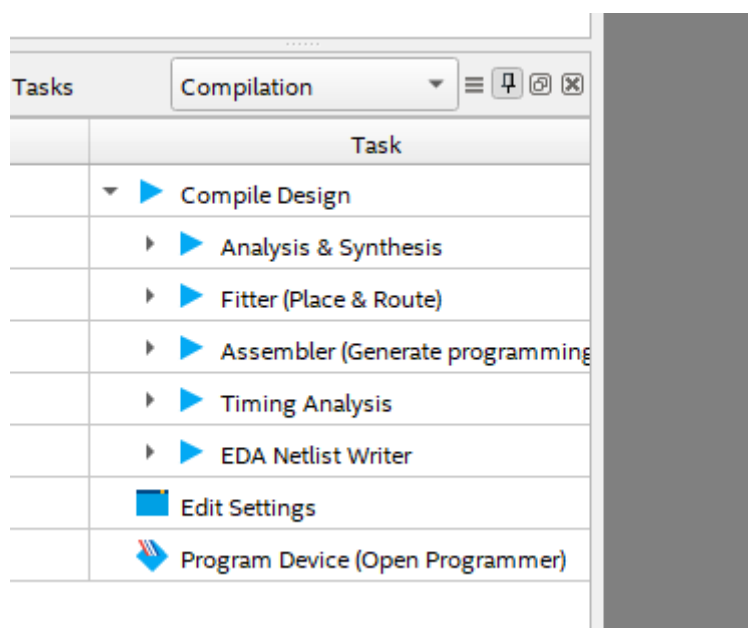


Figure 3.8: Task menu showing the hardware compilation and synthesis process in progress

Allow for the ENTIRE process to be completed, which is indicated by green checkmarks next to all of the six blue play buttons under the “*Task*” menu. A completed and successful run should look something like this:

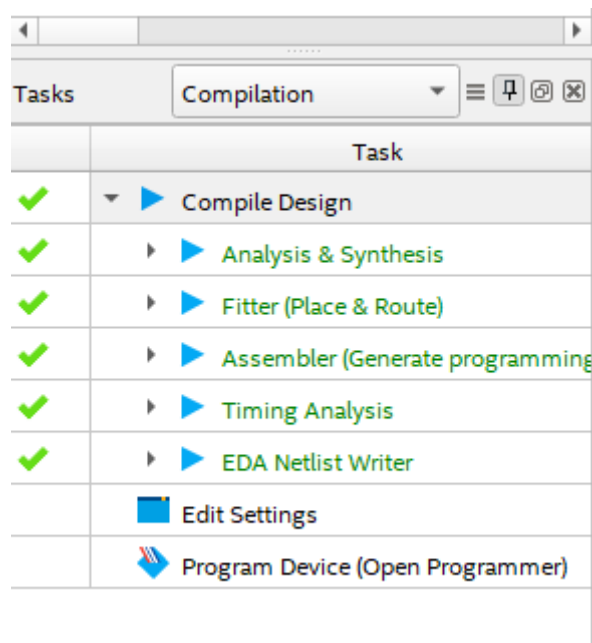


Figure 3.9: Task menu showing a successful compilation with green status indicators

3.1.4 Converting Program to Object File

In order to ensure our program is the one loaded onto the board every time it is plugged into power, it is necessary to convert the compiled program into a “Programming Object File” to be sent to the flash memory on the board’s hardware.

We begin by entering the “File” menu in the toolbar, and selecting “Convert Programming Files...”. This will open the “Convert Programming File” menu. Change the “Mode” setting on the right to “Internal Configuration”, then highlight the “SOF Data” entry in the table at the bottom, followed by the “Add File” button on the right of the table.

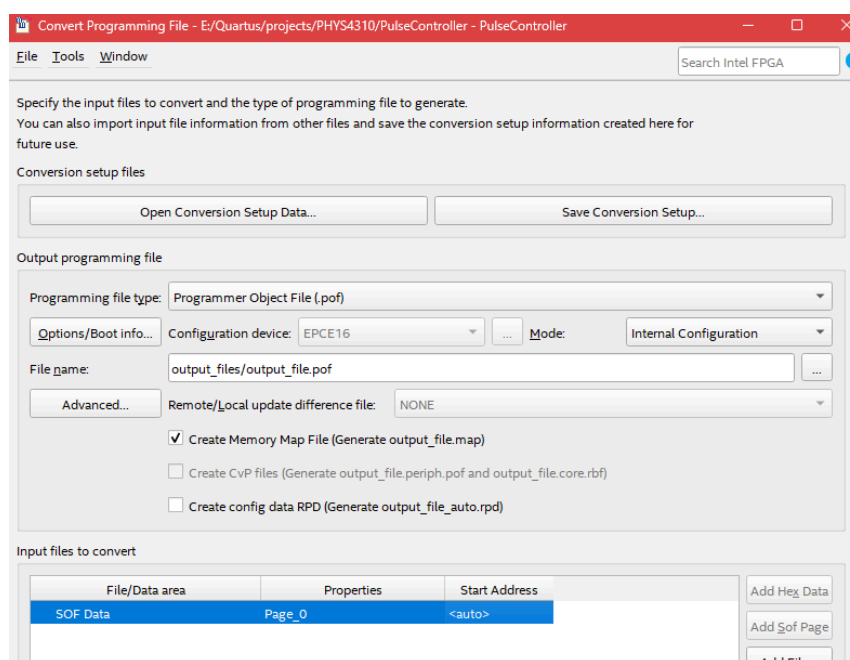


Figure 3.10: “Convert Programming File” menu configured for Internal Configuration mode

In this File Explorer page, navigate to the “*QuartusPrimeProject*” folder again, and then enter the “*output_files*” folder, where you will find a file called “*PulseController.sof*”. Select this file, then press “*Open*”. Now, press the “*Generate*” button at the bottom of the “*Convert Programming File*” window, then close back to the main Quartus menu once the process is complete.

3.1.5 Programming the Flash Memory

Return to the blue checkmark “*Tasks*” menu at the bottom left of the main page, and double click the “*Program Device*” to open the Quartus programmer menu. At this point, ensure that the board’s USB-B programmer cable is plugged into the device you are running Quartus on, then press the “*Hardware Setup*” button at the top left of the programmer.

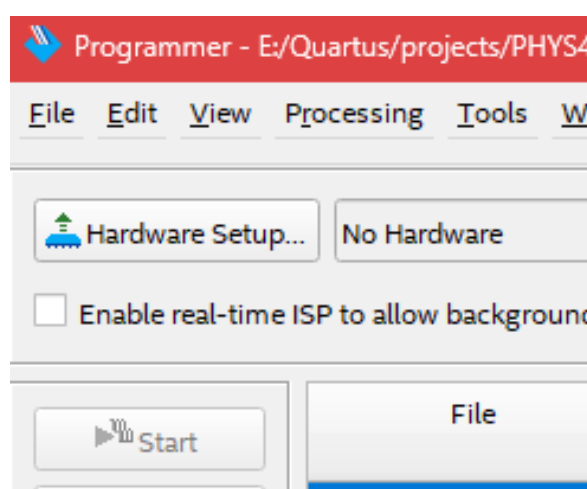


Figure 3.11: Hardware Setup button used to access the Hardware Setup Menu

In the hardware setup menu, enter the “*Currently selected hardware*” dropdown, and select the USB_Blaster option. Close the hardware setup menu.

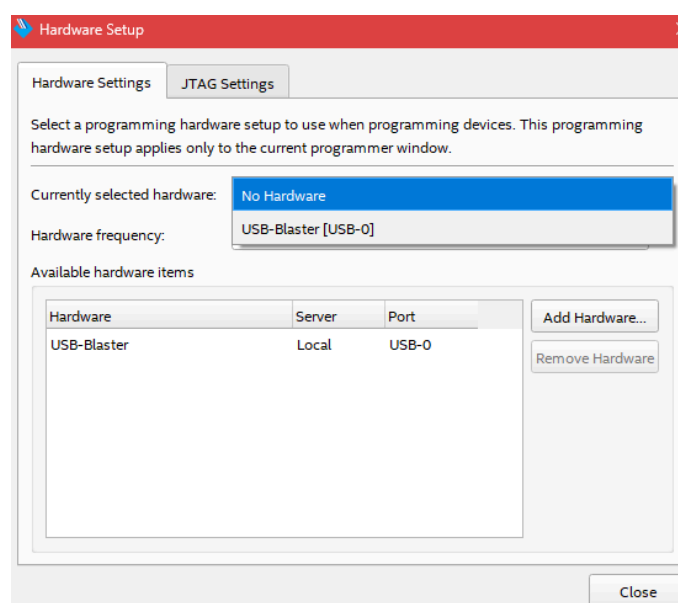


Figure 3.12: Hardware Setup menu used to select the USB-Blaster programmer

Back in the programmer window, press the “Add File...” button on the left, then proceed to navigate back to the “QuartusPrimeProject” folder. Enter the “output_files” directory, select the “PulseController.pof” file, then press “Open”. You will see a new entry come up on the list of files, where you should ensure that the “CFM” option is checked off under the new entry. Right click on the top entry, the one that doesn’t have any “CFM” or “UFM” option under it, and remove it. Below is what the final screen should look like:

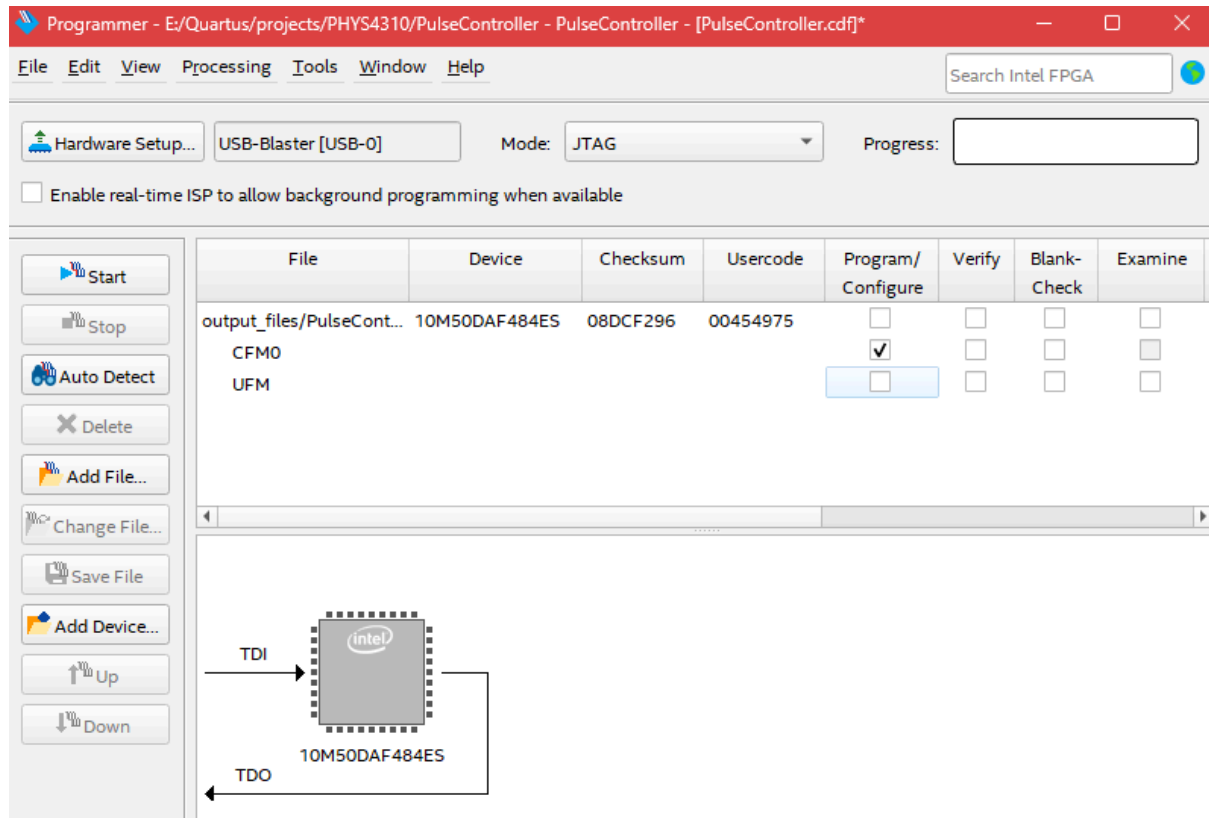


Figure 3.13: Final Programmer configuration screen prior to flashing the CFM memory

Finally, select the “Start” button on the left, and the board should now be fully programmed with the Pulse Generator hardware.

3.2 Computer Software

3.2.1 Pulse Controller Software

The software that interfaces with the hardware comes in the form of a Windows EXE file. You can obtain this file from the [Pulse Generator GitHub](#), following the same download instructions as in **Section 3.1.2**. The EXE file can be found inside of the “PulseGeneratorSoftware” folder, and is called “PulseGenerator.exe”. When you run the program for the first time, you may get a windows security warning. This is normal, as you need to provide the application permission to run on your device. Select the “More Info...” text in the prompt text, then select the “Run Anyway” option. The program should now open without any problems.

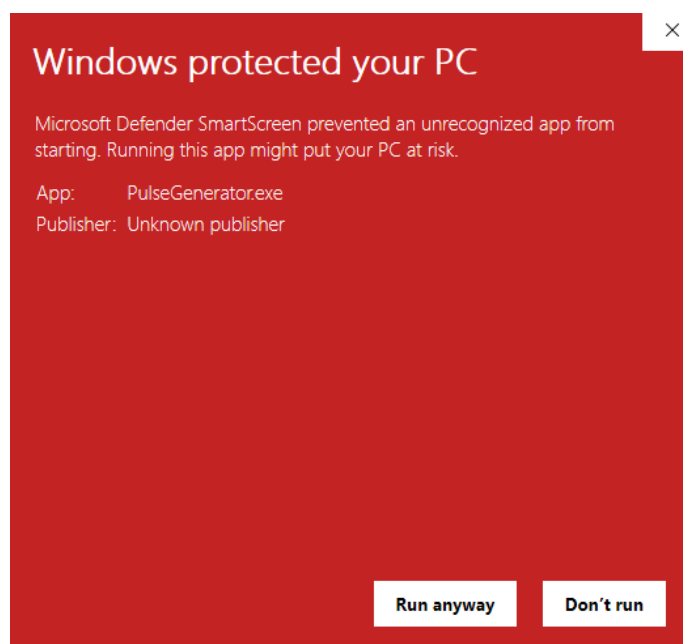


Figure 3.14: Windows Defender SmartScreen warning for an unrecognized publisher

An important note is that the EXE file **MUST** remain inside of the folder you downloaded, you cannot move it, or else it will cease to function properly. If you desire to launch the program from the desktop or some other location, right click the exe in “*File Explorer*”, and select “*Create Shortcut*”. This shortcut can safely be moved anywhere, and is tied to the EXE.

NOTE: If you are on **Windows 11**, you may have to select the “*Show More Options*” when right clicking to see the “*Create Shortcut*” option.

3.2.2 USB Driver

In order for the computer running the software to properly detect the USB interface we will be using to communicate with the hardware, we require a special driver to be installed. To test whether or not you have the driver installed, you can wait until you have completed the physical wiring, and complete the following steps:

1. Plug in the DATA (not programming) USB into the computer
2. Open “*Device Manager*”
3. Expand the “*Ports (COM & LPT)*” option
4. Check for “*Silicon Labs CP210x*”

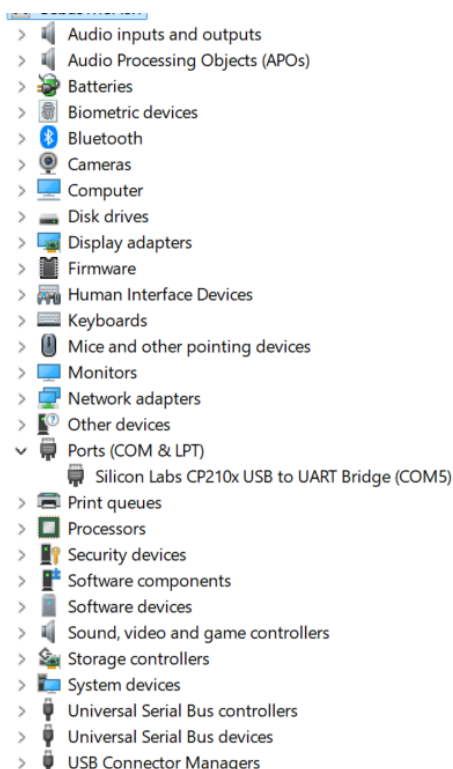


Figure 3.15: Windows 11 Device Manager showing the data USB driver present

If you see the device listed there, then you already have the necessary drivers installed, and are all good to go. If you don't see it, then visit:

<https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers>

Head to the "Downloads" tab, and select "CP210x Windows Drivers". Extract the ZIP folder, then run "CP210xVCPInstaller_x64.exe". Proceed with installation by accepting the ToS, and after it is fully installed, restart your computer.

If not already done, proceed ahead to **Section 3.3** to complete the hardware wiring, and return to this section only to test if the DATA USB is working correctly after the system has been built.

3.3 Physical Wiring

3.3.1 Materials List

The physical implementation of the pulse generator system required several hardware components to interface the FPGA development board with external laboratory equipment and diagnostic indicators. The materials used for the physical wiring and enclosure are summarized in **Figure 3.16**.

Component	Quantity	Description
FPGA Development Board (Intel DE10-Lite)	1	Main processing unit responsible for pulse generation, memory control,

		and UART communication
BNC Panel Connectors	6	Used for external signal connections including trigger input and channel outputs
LED	4	Visual indicators used for system diagnostics including trigger errors, system state, and clock status
Push Button	1	Used as a hardware reset control to return the FPGA system to a known state
Resistors	x8 100Ω, x1 5kΩ	Used as pull-up resistors for reset circuitry and LED current limiting
Hookup Wires / Jumper Wires	~x50-75 (as needed Male to Male, Female to Male, and Female to Male)	Used to connect board I/O pins to external connectors and indicators
CP210x Series USB to TTL Adapter	1	Used to interface the communication between the computer software and FPGA hardware (any CP210 series adapter should work)
Electrical Tape	1 roll	Used to secure and isolate wiring in the module
Hot Glue	As needed	Use to secure electrical components in the module
3D Printer (Optional, but highly recommended)	N/A	Needed to print the bottom casing for the FPGA (cardboard or other substitute may be used, refer to Section 3.3.6)
Label Printer (Optional, but highly recommended)	N/A	Needed to print the labels for the ports (other substitute may be used, refer to Section 3.3.6)
Enclosure / Chassis (DIMENSIONS)	1	Mechanical housing used to protect the electronics and mount the external connectors.

Figure 3.16: Comprehensive list of components required to assemble the Pulse Generator

3.3.2 Chassis

The electronics were mounted inside a protective chassis designed to provide both structural stability and convenient access to external signal connections. The enclosure serves several purposes: mechanical protection of the FPGA board, organization of the external connectors, and safe routing of signal wiring.

The front panel of the chassis contains several BNC connectors corresponding to the trigger input and pulse output channels. These connectors allow the system to interface with standard laboratory equipment such as oscilloscopes, function generators, and other test instruments using coaxial cables.

Additional panel space is allocated for status LEDs and a reset push button, allowing quick visual inspection of system state and manual recovery from error conditions without requiring reprogramming of the FPGA.

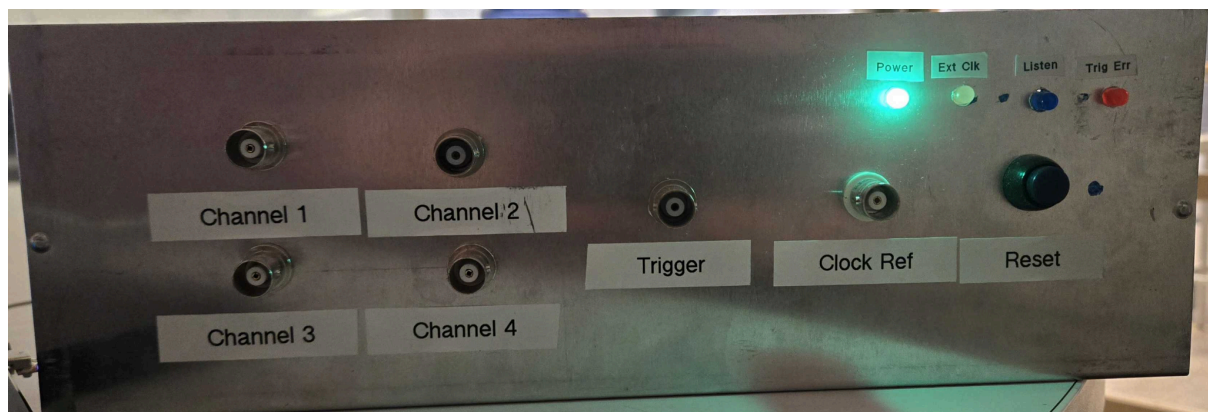


Figure 3.17: Example front panel interface for the chassis

The exact dimensions of holes and spacing between elements is entirely dependent on the target aesthetic, and causes little to no functional consequences. It is important to note that the BNC port holes should be measured to be precisely the size of the threads, to allow for proper fastening and a snug fit that will prevent the port from coming loose during continuous use.

An additional hole may be drilled on either side or back of the unit for routing the USB cable out at the end of construction. The final decision is again up to the builder, in this case, a hole was drilled on the right side of the unit.

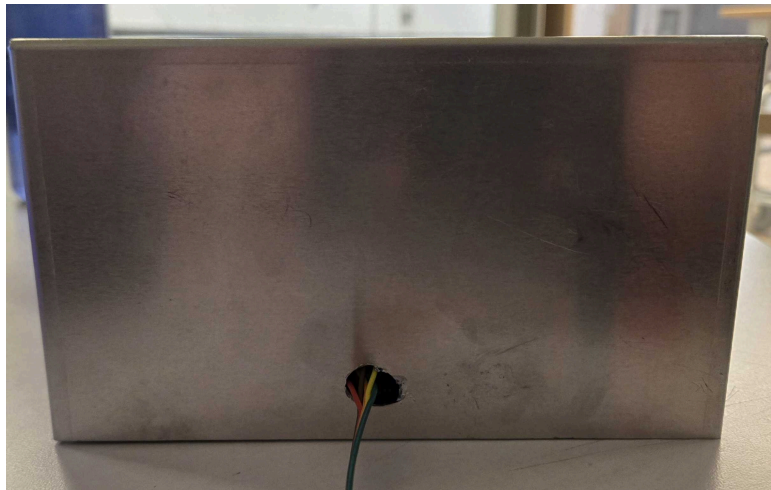


Figure 3.18: Example hole for data USB cable on side of unit

3.3.3 BNC I/O to Board

3.3.4 LEDs and Reset Button to Board

3.3.5 USB to Board

3.3.6 Final Touches

Board mount, gluing board mount, gluing USB to box to avoid excessive tugging, adding labels,

4. Usage and Basic Troubleshooting

The Pulse Generator software serves as a user-friendly interface to program the hardware that we flashed onto the FPGA. It will serve to provide instructions to the board while it is in

its configuration state, as well as provide controls to swap it in and out of its behavioral domain.

4.1 Signal and Project Management

The software is designed to allow users to define complex pulse patterns through a modular “*Signal*” architecture. Experiments can be characterized by “*Projects*”, where “*Signals*” can be added and their properties modified. When the application is first launched, you are greeted with an empty project.

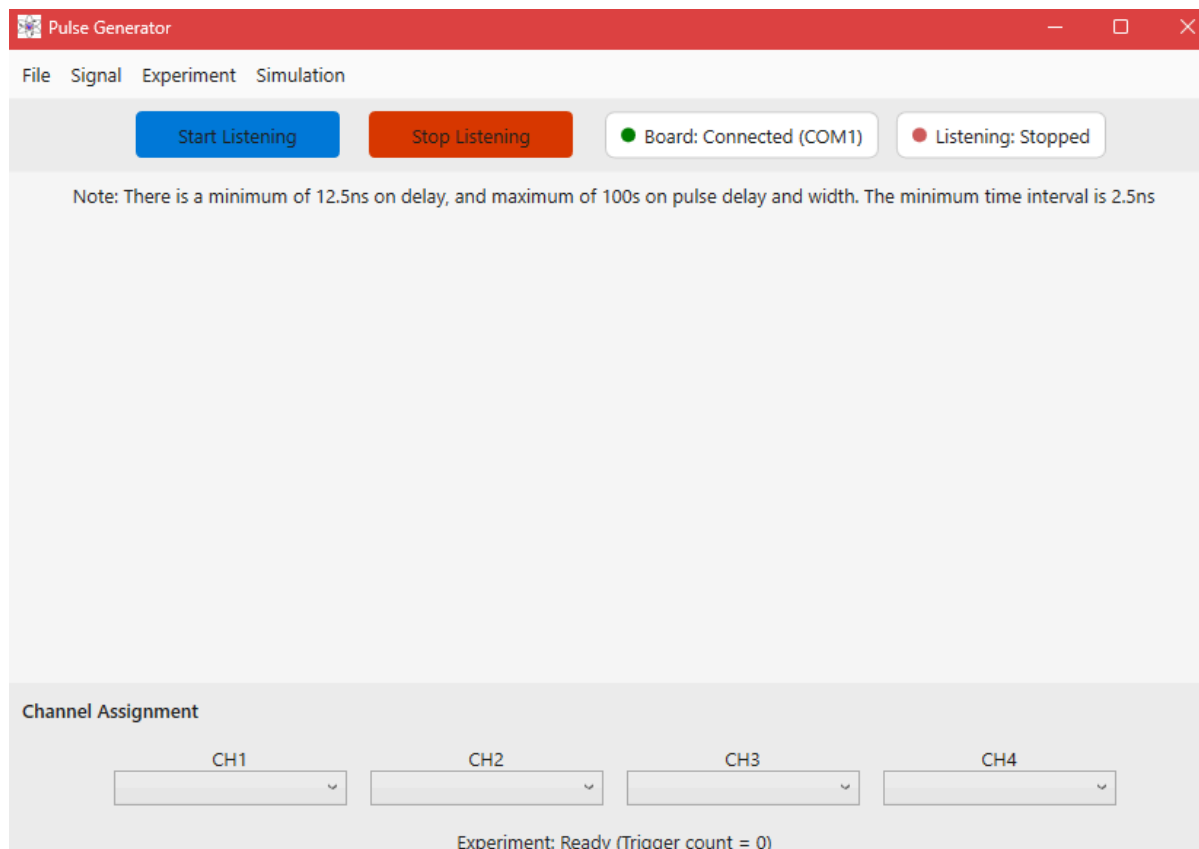


Figure 3.15: Empty project state upon initial launch of the Pulse Generator software

To begin adding signals, select the “*Signal*” option, and the “*New Signal*” submenu option. This will add a new signal to the project, shown through the newly added signal module in the project area. The name of the signal can be changed, as well as its programmed delay and width. Up to 10 signals may be added per project, all with independent names, delays, and widths. Under the “*Signal*” submenu option, there exists the function to “*Import*” and “*Export*” individual signals as files as well, in case you want to share signals between projects, or want to store specific timings for future use. A similar functionality exists for projects, under the “*File*” option, where you can “*Save*” your current project, “*Save As*” your current project into a new file, or “*Export*” your current project into a new file. The “*Save As*” and “*Export*” options are functionally identical as of the current state of the software, while “*Save*” will save the project to its current file if it was imported, or prompt you to define a new file if the project has not yet been attributed to a file. A new project can always be created under the “*File*” menu with the “*New Project*” option, but take care to ensure your current project is saved before overwriting it.

4.2 Channel Routing

Once you have defined your project and signals, you can then assign your signals to the designed four output channels using the dropdown menus at the bottom of the application. You are free to leave them undefined to inform the software you want no output from these channels on start. The same signal may be outputted on more than one channel as well.

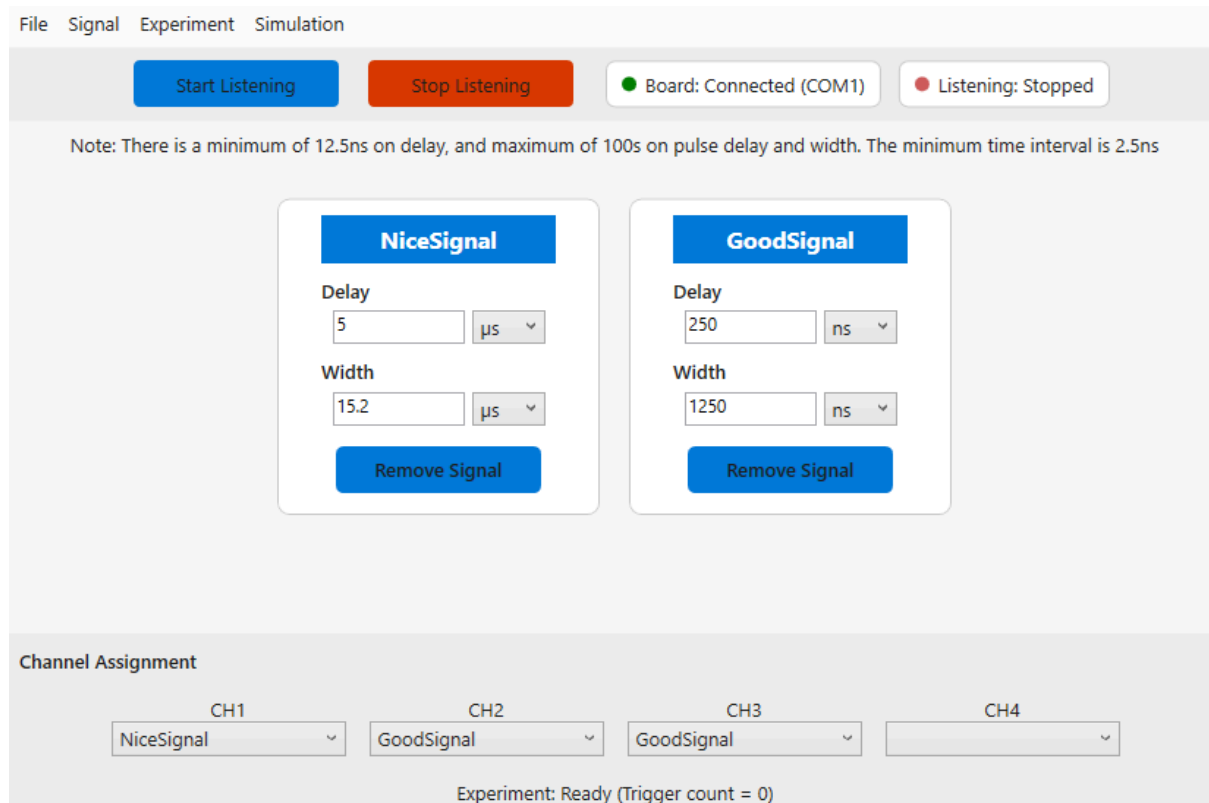


Figure 3.16: Software interface with multiple active signals and mapped channel assignments

Removing a signal will automatically remove the signal from all output channels, but regardless ensure that all your channels are properly defined signals to avoid any undefined behavior while running the program.

4.3 Running the Experiment

After the signals, project, and channel outputs are all programmed, the experiment can now be run. At this point, ensure the board is plugged into the computer using the provided USB connection, and that the software recognizes the board, indicated by the “*Board: Connected*” with the green indicator when properly connected. Then, press the “*Start Listening*” button, which will switch the board into its **behavioral domain**, as described in **Section 1** of the report. Once the programming is done, the “*Listening*” indicator will turn green, and the board is ready to receive a trig signal through its input. If at any point the signal values want to be modified, you must press the “*Stop Listening*” button, and restart the listening to apply the new delay changes.

4.4 Basic Troubleshooting

If the program is failing to recognize the board as a USB device, a very common cause of this issue is a lack of power delivery by the USB. The board is powered by the USB plugged into the computer, so ensure that the system is capable of powering the unit, which can be checked by the on board power LED on the chassis

Another common issue may be caused by having multiple USB devices connected to the device being used to program the board. The software may incorrectly detect another USB device as the board, and attempt to establish a connection with the device in a protocol it doesn't understand, possibly manifesting itself as some sort of error when attempting to program the board, even if the “*Board*” indicator shows up as “*Connected*”. For this reason, avoid multiple USB devices, especially those that transmit data connected to the computer at the same time.

For all other errors that may be encountered, a good catch all is the “*System Reset*” button on the chassis, which will reset the system state internally. An even safer bet is to power cycle the system entirely by disconnecting and reconnecting the generator's USB to the computer being used to program it. Closing the Pulse Generator software while doing this further increases the chance of any potential desync being resolved entirely.

As a last resort, reflashing the hardware, or checking all connections with a multimeter may be the only way to fish out more specific issues. It is strongly recommended to NOT modify the hardware files, as every component plays a very specific role, but in the case that some change needs to be made, or some experimental change is to be added, always refer back to the [Pulse Generator GitHub](#) for a clean working backup of the hardware files.

5. Further Discussion

5.1 Future Hardware

5.1.1 Full Portability

Future iterations of the Pulse Generator design could achieve full standalone operation without the need of a tethered computer. Instead of relying on USB for power and configuration, an internal power supply can be implemented with a 120V AC to DC converter. Configuration can be facilitated by an array of physical controls such as potentiometers and OLED displays integrated directly into the chassis, removing the need

for a software interface. The necessity for such a change would be entirely dependent on the use case of the unit, as the flexibility of software updates may outweigh the convenience of a standalone system.

5.1.2 Sub Cycle Delays

While the current system is limited to a timing resolution of 2.5ns based on the clock period, sub-cycle precision could be achieved by utilizing “hardware taps” or delay lines. This architecture would use the current 400MHz clock for coarse timing, then feed the output signal through a series of transparent delay modules to achieve further precision. For example, if we have a series of 1000 delay modules, each with a propagation delay of 0.001ns, and a delay of 5.62ns is desired, a coarse delay of 2 clock cycles would be set, and the output would be fed into the series of transparent delay modules, with the tap at position 620 to be enabled. The feasibility of such an application, considering the inherent rise and fall time of the pulse edges or tap thermal jitter may render it ineffective, but the technology warrants some potential investigation if other pulse artifacts can be mitigated.

5.2 Future Software

5.2.1 Experimental Modifiers

The next step for the software client involves implementing the experimental model system. These experiments would allow modifiers that change pulse patterns automatically over time over the course of an experiment. For example, linearly increasing the pulse delay over 100 trigger cycles in a single experiment. Additionally, allowing the software to be compatible with other industry-standard signal formats such as LabView could allow for easier adaptation into existing physics laboratory systems.

5.2.2 Simulation Pulse Previews

Expanding the simulation capabilities of the software will remain a priority. Providing a basic preview of the full waveform simulation over multiple triggers, especially with experimental modifiers is vital to allow for easy validation of the inputted signal parameters.

5.3 Future Architecture

5.3.1 Concurrent Programming

Currently, the system must enter the “Stop Listening” state to update the delay or width values in the internal countdown hardware registers. Shifting the architecture to allow for concurrent programming would allow new modifiers to be flashed live to the system while it remains in the behavioral domain. This would enable real time updates to the pulse timings without interrupting the live experiment currently being executed.

5.3.2 Sine Wave Rubidium Reference Clock

FPGAs inherently require digital signals as input to operate properly, and test runs with a sine wave reference clock demonstrated undefined behavior. To support high precision rubidium atomic clocks that output an analog sine wave, a sine to square wave digitizer

would need to be implemented that doesn't introduce excessive jitter, reduced resolution, and produces sharp rise and fall times on clock edges.

6. References

- [1] GeeksforGeeks, "Central Processing Unit (CPU)," *GeeksforGeeks*, Jun. 14, 2021.
<https://www.geeksforgeeks.org/computer-science-fundamentals/central-processing-unit-cpu/>
- [2] A FPGA Project with MYRIO - Scientific Figure on ResearchGate. Available from:
https://www.researchgate.net/figure/nternal-structure-of-FPGA_fig1_321748886 [accessed 9 Feb 2026]